

MS - repetition - frac_included - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4207 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```

PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)

# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite

# get counts of syllable components in different serial positions
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))

syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))

kable(syll_comp_dist_N)

```

pos_factor	O	P	V	1	S	total
1	531	29	128	NA	NA	688
2	61	NA	424	96	107	688
3	305	NA	165	204	14	688
4	295	NA	231	64	37	627
5	221	NA	208	71	35	535
6	203	1	129	69	21	423
7	169	NA	100	26	18	313
8	86	NA	52	25	4	167
9	72	NA	2	NA	4	78

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7718023	0.0421512	0.1860465	NA	NA	688
2	0.0886628	NA	0.6162791	0.1395349	0.1555233	688
3	0.4433140	NA	0.2398256	0.2965116	0.0203488	688
4	0.4704944	NA	0.3684211	0.1020734	0.0590112	627
5	0.4130841	NA	0.3887850	0.1327103	0.0654206	535
6	0.4799054	0.0023641	0.3049645	0.1631206	0.0496454	423
7	0.5399361	NA	0.3194888	0.0830671	0.0575080	313
8	0.5149701	NA	0.3113772	0.1497006	0.0239521	167
9	0.9230769	NA	0.0256410	NA	0.0512821	78

```

# get percentages only from summary table
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")

# pivot to long format for plotting
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))

# plot satellite and coda occurrences across position

```

```
syll_comp_plot <- ggplot(syll_comp_plot_data,
  aes(x=pos,y=percent,group=syll_component,
      color=syll_component,
      linetype = syll_component,
      shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=s
```

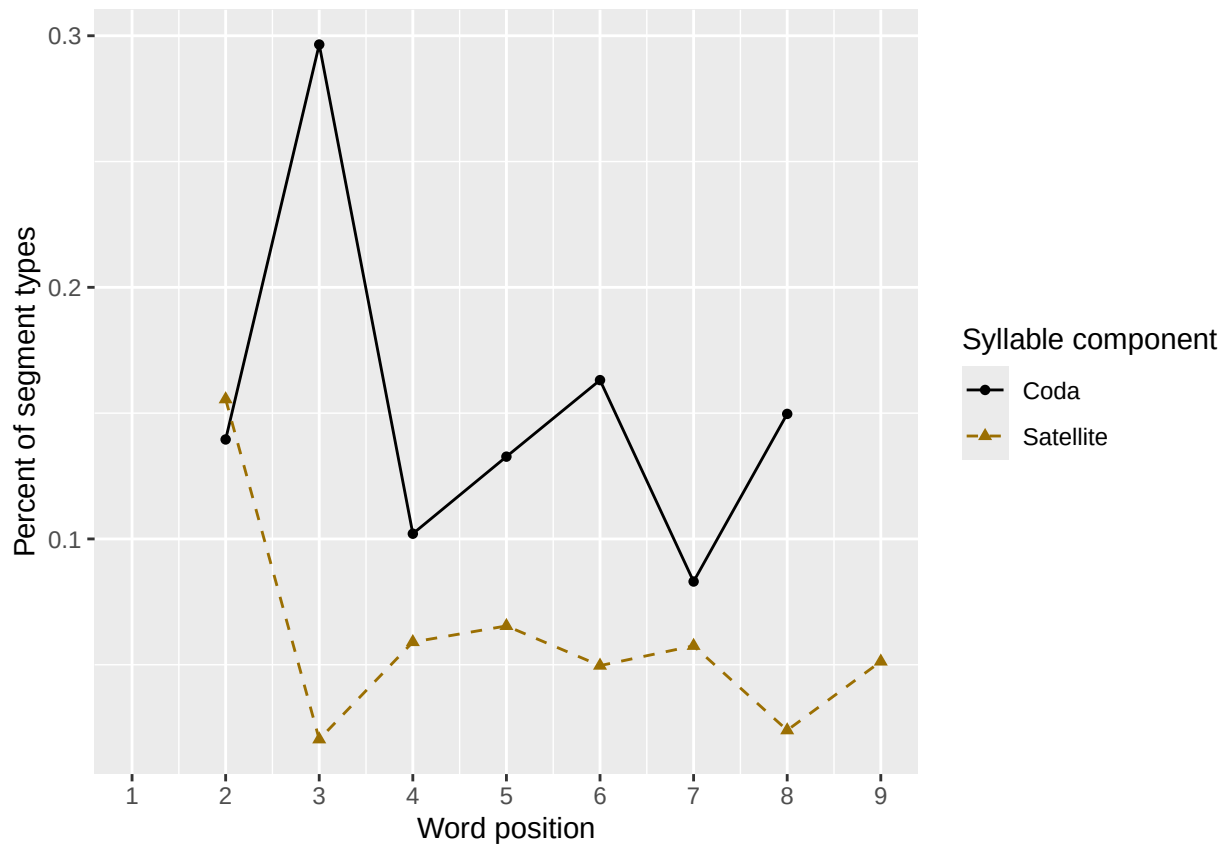
```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```



```
# len/pos table
```

```
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
```

```
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
```

```
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```
min_preserved <- min_preserved_raw - (0.1 * preserved_range)
```

```
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
```

```
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen `1` `2` `3` `4` `5` `6` `7` `8` `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.984 0.984 0.984 NA      NA      NA      NA      NA      NA
## 2      5 0.913 0.957 0.940 0.940 NA      NA      NA      NA      NA
## 3      6 0.982 0.964 0.942 0.955 0.920 NA      NA      NA      NA
## 4      7 0.964 0.932 0.909 0.927 0.932 0.927 NA      NA      NA
## 5      8 0.947 0.901 0.914 0.817 0.844 0.828 0.823 NA      NA
## 6      9 0.938 0.843 0.831 0.770 0.824 0.777 0.760 0.751 NA
## 7     10 0.957 0.915 0.923 0.880 0.850 0.821 0.806 0.838 0.829
```

```
# len/pos table
```

```
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))
```

```
pos_len_N_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen `1` `2` `3` `4` `5` `6` `7` `8` `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    61    61    61    NA    NA    NA    NA    NA    NA
## 2      5    92    92    92    92    NA    NA    NA    NA    NA
## 3      6   112   112   112   112   112    NA    NA    NA    NA
## 4      7   110   110   110   110   110   110    NA    NA    NA
## 5      8   146   146   146   146   146   146   146    NA    NA
## 6      9    89    89    89    89    89    89    89    89    NA
## 7     10    78    78    78    78    78    78    78    78    78
```

```
obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")
```

```
pred_linetypes <- "dashed"
```

```
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
```

```
pos_len_summary$pos <- factor(pos_len_summary$pos)
```

```
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
```

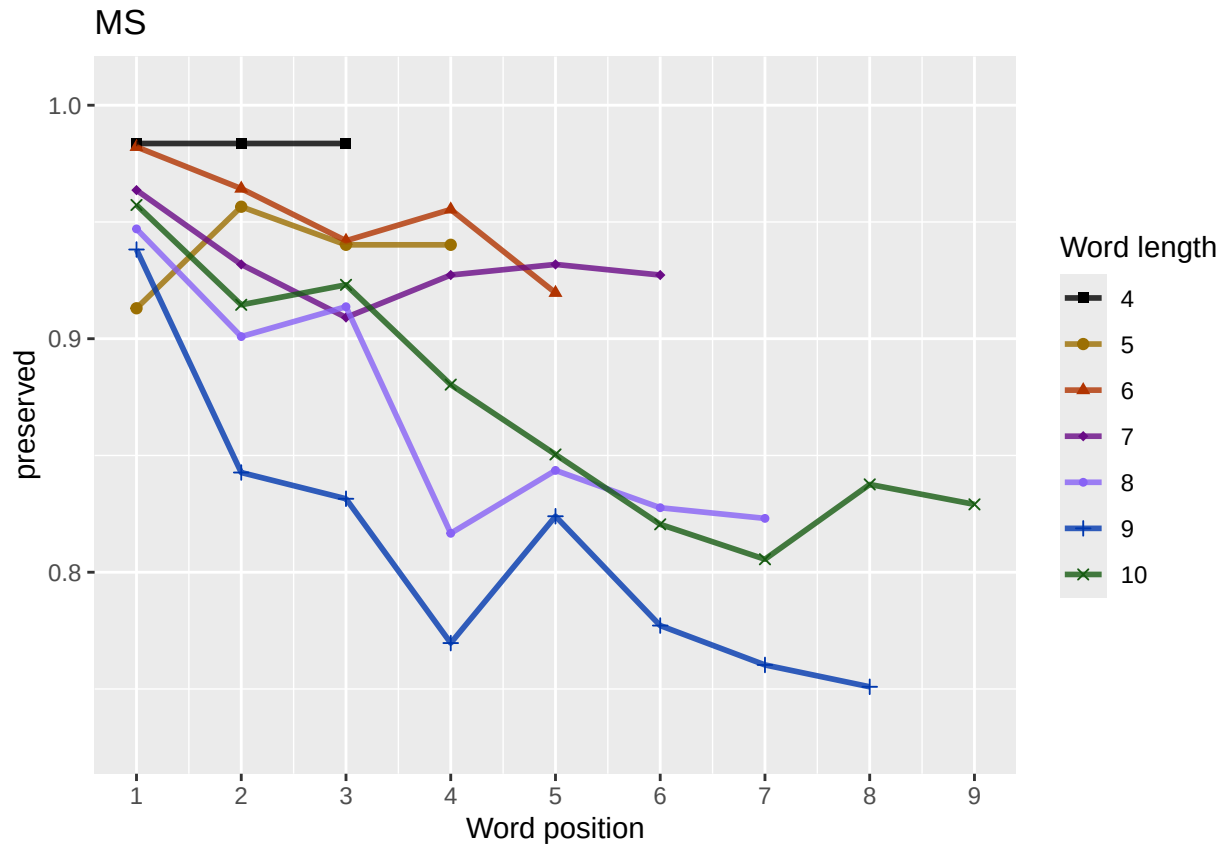
```
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
                      "preserved ~ stimlen",
                      "preserved ~ pos",
                      "preserved ~ stimlen + pos",
                      "preserved ~ stimlen*pos",
                      "preserved ~ I(pos^2)+pos",
                      "preserved ~ stimlen + I(pos^2) + pos",
                      "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
```

```

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      5.01285      -0.21958      0.03131      -0.44263
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4203 Residual
## Null Deviance:      2806
## Residual Deviance: 2673 AIC: 2858
## log likelihood: -1336.702
## Nagelkerke R2: 0.06385472
## % pres/err predicted correctly: -782.3133
## % of predictable range [ (model-null)/(1-null) ]: 0.03333209
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      3.01491      0.01179      -0.12464      0.80620      0.01733      -0.01733
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4201 Residual
## Null Deviance:      2806
## Residual Deviance: 2669 AIC: 2858
## log likelihood: -1334.405
## Nagelkerke R2: 0.06602675
## % pres/err predicted correctly: -781.1732
## % of predictable range [ (model-null)/(1-null) ]: 0.0347391
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      5.18426      -0.29899      -0.39068      0.02698
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4203 Residual
## Null Deviance:      2806
## Residual Deviance: 2680 AIC: 2866
## log likelihood: -1340.082
## Nagelkerke R2: 0.06065364
## % pres/err predicted correctly: -783.5009
## % of predictable range [ (model-null)/(1-null) ]: 0.03186653

```

```

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.3317      -0.1996      -0.1534
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4204 Residual
## Null Deviance:      2806
## Residual Deviance: 2683 AIC: 2868
## log likelihood: -1341.346
## Nagelkerke R2: 0.05945542
## % pres/err predicted correctly: -783.6142
## % of predictable range [ (model-null)/(1-null) ]: 0.03172665
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.36526      0.02025      -0.40714
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4204 Residual
## Null Deviance:      2806
## Residual Deviance: 2710 AIC: 2898
## log likelihood: -1355.226
## Nagelkerke R2: 0.04624884
## % pres/err predicted correctly: -789.881
## % of predictable range [ (model-null)/(1-null) ]: 0.02399298
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.0116      -0.2164
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2714 AIC: 2902
## log likelihood: -1357.219
## Nagelkerke R2: 0.044345
## % pres/err predicted correctly: -790.0866
## % of predictable range [ (model-null)/(1-null) ]: 0.02373923
## *****
## model index: 2
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.3544      -0.2837
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2724 AIC: 2904
## log likelihood: -1361.802
## Nagelkerke R2:  0.03996133
## % pres/err predicted correctly: -792.8356
## % of predictable range [ (model-null)/(1-null) ]:  0.02034676
## *****
## model index:  1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.1
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4206 Residual
## Null Deviance:      2806
## Residual Deviance: 2806 AIC: 2990
## log likelihood: -1403.123
## Nagelkerke R2:  0
## % pres/err predicted correctly: -809.323
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                        AIC=LPRes$AIC,
                        row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                            "_len_pos_model_summary.csv"),row.names = FALSE)
kable(LPAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	2857.901	0.000000	0.000000	0.501788	0.206385	5.012852	-	-	NA	0.0313124	NA
							0.2195777	0.4426336			
preserved ~ stimlen * (I(pos^2) + pos)	2857.901	0.0601862	0.970355	0.248691	0.070660	2.870149	0.011793	0.08061966	-	-	0.0173297
									0.1413130	0.1246396	
preserved ~ stimlen * pos	2866.368	8.464260	0.014520	0.007280	0.070606	5.36184263	-	-	0.0269816	NA	NA
							0.2989929	0.3906788			
preserved ~ stimlen + pos	2867.561	0.657567	0.007996	0.004010	0.059451	4.331697	-	-	NA	NA	NA
							0.1996427	0.1534431			
preserved ~ I(pos^2) + pos	2898.465	0.561146	0.000000	0.000000	0.004624	8.365260	NA	-	NA	0.0202477	NA
								0.4071437			
preserved ~ pos	2901.708	3.804198	0.000000	0.000000	0.004434	3.011649	NA	-	NA	NA	NA
								0.2164271			
preserved ~ stimlen	2903.792	5.888634	0.000000	0.000000	0.003996	4.3354417	-	NA	NA	NA	NA
							0.2836826				
preserved ~ 1	2989.971	32.06756	0.000000	0.000000	0.000000	0.0099764	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen + I(pos^2) + pos"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",  
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)      stimlen      I(pos^2)          pos  
##      5.01285      -0.21958      0.03131      -0.44263
```

```
##
```

```
## Degrees of Freedom: 4206 Total (i.e. Null); 4203 Residual
```

```
## Null Deviance: 2806
```

```
## Residual Deviance: 2673 AIC: 2858
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],  
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.  
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)  
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups:   stimlen [7]
```

```
##   stimlen  `1`  `2`  `3`  `4`  `5`  `6`  `7`  `8`  `9`  
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

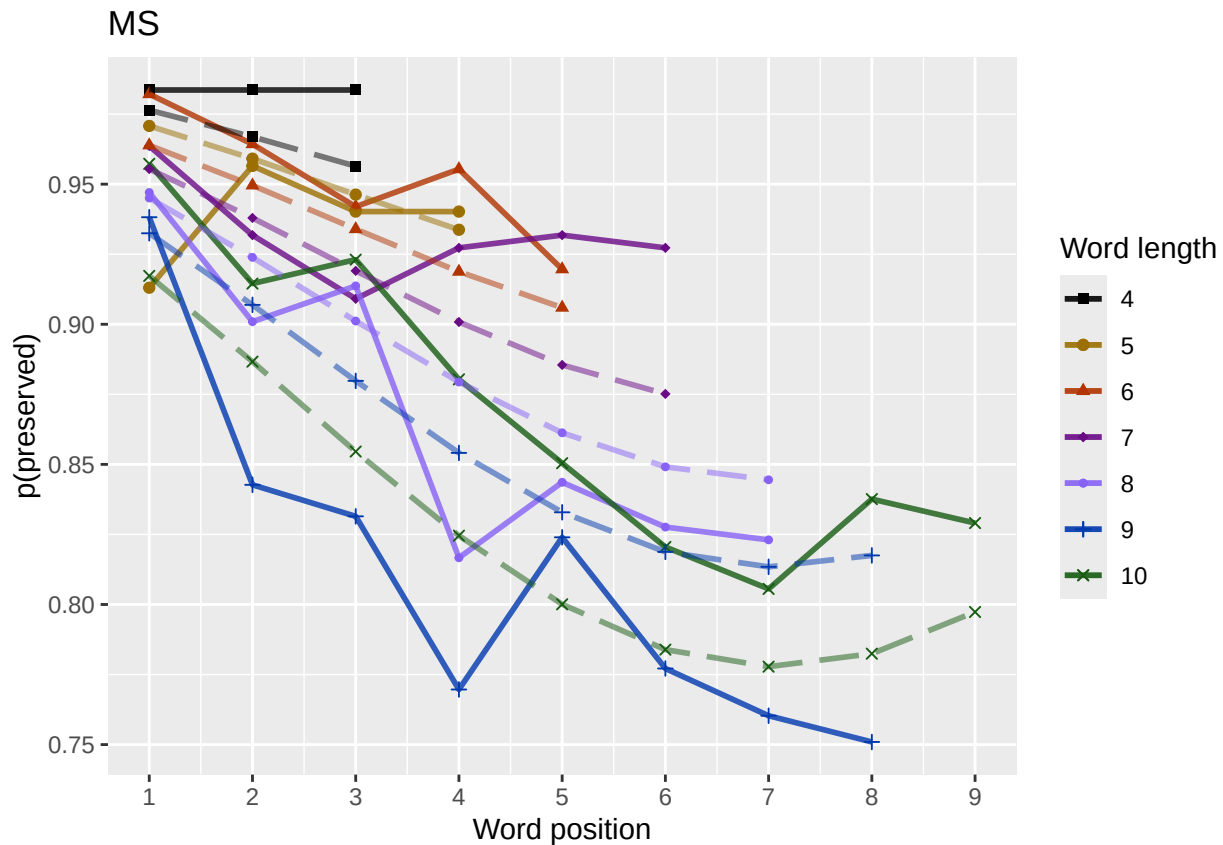
```
## 1      4 0.976 0.967 0.956 NA      NA      NA      NA      NA      NA
## 2      5 0.971 0.959 0.946 0.934 NA      NA      NA      NA      NA
## 3      6 0.964 0.950 0.934 0.919 0.906 NA      NA      NA      NA
## 4      7 0.955 0.938 0.919 0.901 0.885 0.875 NA      NA      NA
## 5      8 0.945 0.924 0.901 0.879 0.861 0.849 0.845 NA      NA
## 6      9 0.932 0.907 0.880 0.854 0.833 0.819 0.813 0.818 NA
## 7     10 0.917 0.887 0.855 0.825 0.800 0.784 0.778 0.782 0.797
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient, " - LP Fitted"))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient, " - LP Fitted"))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot)
```



length and position without fragments to see if this changes position² influence

*# first number responses, then count resp with fragments - below we will eliminate fragments
and re-run models*

number responses

```
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array
```

count responses with fragments

```
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag
```

```
## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      66   688
```

```

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.
print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 66 / 688 = 9.59 percent"
write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos
## 5.28213 -0.26175 0.04795 -0.35071
##
## Degrees of Freedom: 3969 Total (i.e. Null); 3966 Residual
## Null Deviance: 1699
## Residual Deviance: 1660 AIC: 1804
## log likelihood: -829.9947
## Nagelkerke R2: 0.02840694
## % pres/err predicted correctly: -427.3655
## % of predictable range [ (model-null)/(1-null) ]: 0.01105076
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos stimlen:I(pos^2) stimlen:I(pos^2):pos
## 2.97838 0.02243 -0.10298 0.99393 0.01798 -0.00000
##
## Degrees of Freedom: 3969 Total (i.e. Null); 3964 Residual
## Null Deviance: 1699

```

```

## Residual Deviance: 1656  AIC: 1805
## log likelihood:  -828.204
## Nagelkerke R2:  0.03097061
## % pres/err predicted correctly:  -427.0995
## % of predictable range [ (model-null)/(1-null) ]:  0.01166493
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.48023      -0.24015       0.05056
##
## Degrees of Freedom: 3969 Total (i.e. Null);  3967 Residual
## Null Deviance:      1699
## Residual Deviance: 1670  AIC: 1814
## log likelihood:  -834.921
## Nagelkerke R2:  0.02134196
## % pres/err predicted correctly:  -428.6053
## % of predictable range [ (model-null)/(1-null) ]:  0.008188477
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.4932      -0.2177
##
## Degrees of Freedom: 3969 Total (i.e. Null);  3968 Residual
## Null Deviance:      1699
## Residual Deviance: 1672  AIC: 1815
## log likelihood:  -836.1103
## Nagelkerke R2:  0.01963374
## % pres/err predicted correctly:  -428.8732
## % of predictable range [ (model-null)/(1-null) ]:  0.007570072
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      4.83260      -0.28140      -0.06532       0.01327
##
## Degrees of Freedom: 3969 Total (i.e. Null);  3966 Residual
## Null Deviance:      1699
## Residual Deviance: 1670  AIC: 1815
## log likelihood:  -834.7607
## Nagelkerke R2:  0.02157214

```

```

## % pres/err predicted correctly: -428.566
## % of predictable range [ (model-null)/(1-null) ]: 0.008279166
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 3.2864 0.0339 -0.2991
##
## Degrees of Freedom: 3969 Total (i.e. Null); 3967 Residual
## Null Deviance: 1699
## Residual Deviance: 1694 AIC: 1840
## log likelihood: -847.1558
## Nagelkerke R2: 0.003719732
## % pres/err predicted correctly: -431.654
## % of predictable range [ (model-null)/(1-null) ]: 0.00115008
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.782
##
## Degrees of Freedom: 3969 Total (i.e. Null); 3969 Residual
## Null Deviance: 1699
## Residual Deviance: 1699 AIC: 1842
## log likelihood: -849.7288
## Nagelkerke R2: 6.376242e-16
## % pres/err predicted correctly: -432.1522
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 2.83096 -0.01323
##
## Degrees of Freedom: 3969 Total (i.e. Null); 3968 Residual
## Null Deviance: 1699
## Residual Deviance: 1699 AIC: 1844
## log likelihood: -849.6441
## Nagelkerke R2: 0.0001225123
## % pres/err predicted correctly: -432.1201
## % of predictable range [ (model-null)/(1-null) ]: 7.390891e-05
## *****

```

```

NoFragBestLPModel<-NoFrag_LPres$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPres$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPres$Model,
                                AIC=NoFrag_LPres$AIC,
                                row.names = NoFrag_LPres$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPres$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPres$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	1803.923	0.000000	1.000000	0.061833	0.082840	59282128	-	-	NA	0.0479517	NA
							0.261752	183507092			
preserved ~ stimlen * (I(pos^2) + pos)	1804.941	1.013072	0.602579	0.237259	0.070309	7069783790	0.224342	9939326	-	-	0.0179843
									0.1628019	10.1029819	
preserved ~ stimlen + pos	1813.960	0.032360	0.006629	0.004099	0.052134	20480226	-	0.0505583	NA	NA	NA
							0.2401457				
preserved ~ stimlen	1814.585	0.657588	0.004849	0.002998	0.019633	7493168	-	NA	NA	NA	NA
							0.2176988				
preserved ~ stimlen * pos	1815.431	1.505056	0.003174	0.001963	0.012157	21832605	-	-	0.0132688	NA	NA
							0.2814008	0.0653203			
preserved ~ I(pos^2) + pos	1839.635	5.708103	0.000000	0.000000	0.000371	97286421	NA	-	NA	0.0338970	NA
								0.2991051			
preserved ~ 1	1842.153	8.223755	0.000000	0.000000	0.000000	20781554	NA	NA	NA	NA	NA
preserved ~ pos	1843.974	0.051961	0.000000	0.000000	0.000125	530964	NA	-	NA	NA	NA
								0.0132285			

```

# plot no fragment data

NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
                                                       NULL,palette_values=palette_values)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

# len/pos table
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

```

nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted_pos_len)
nofrag_fitted_pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.981 0.976 0.974 NA    NA    NA    NA    NA    NA
## 2      5 0.975 0.970 0.966 0.966 NA    NA    NA    NA    NA
## 3      6 0.968 0.961 0.957 0.956 0.959 NA    NA    NA    NA
## 4      7 0.959 0.950 0.944 0.943 0.948 0.956 NA    NA    NA
## 5      8 0.947 0.936 0.929 0.928 0.933 0.943 0.956 NA    NA
## 6      9 0.932 0.918 0.909 0.908 0.915 0.927 0.944 0.960 NA
## 7     10 0.914 0.896 0.885 0.884 0.892 0.908 0.928 0.949 0.967

```

```

fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen)) +
#   geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("NoFragData",
#   "LPFitted",
#   NULL,
#   palette_values,
#   shape_values,
#   obs_linetypes,
#   pred_linetypes = c("longdash")
#   )

```

```

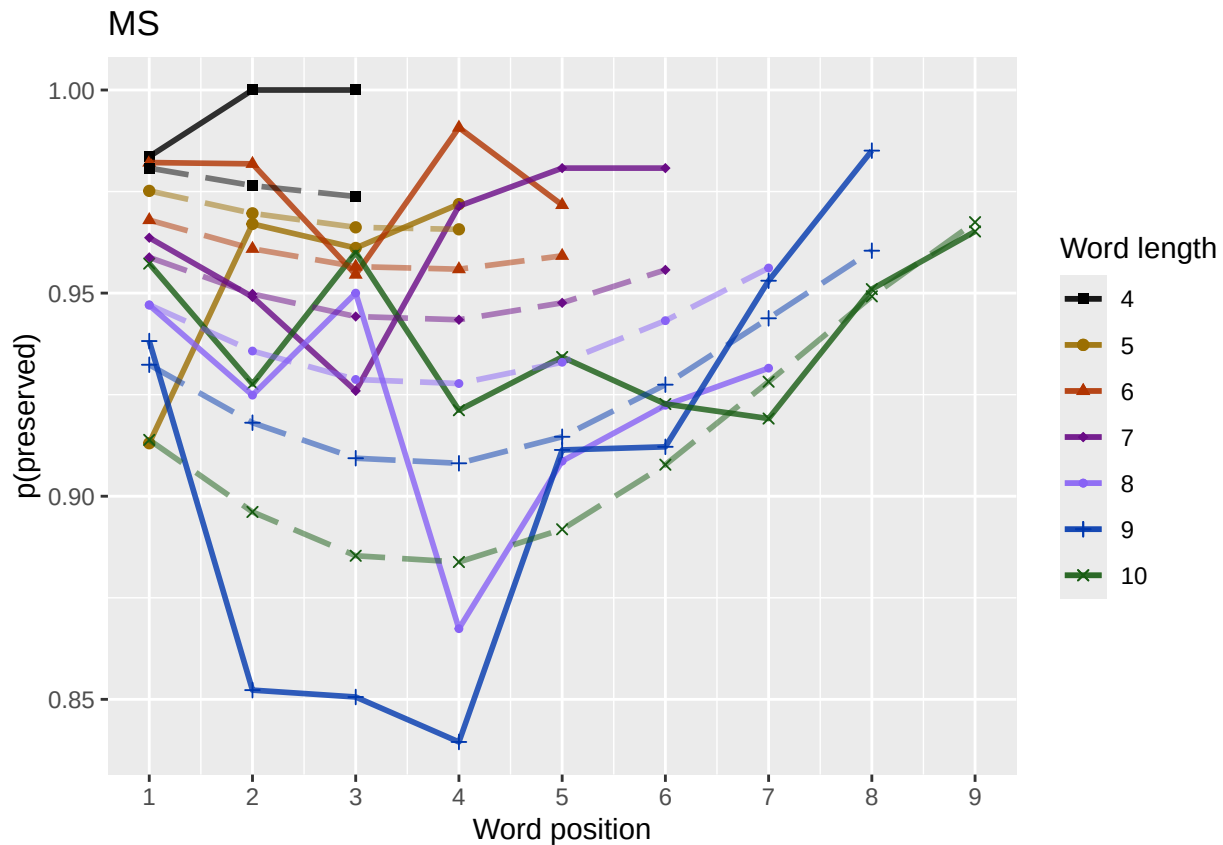
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

```

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot

```



back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.73 - 1.01"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.01745796
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] -0.01601546
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                              2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

CurrentLabel<-"Average upward change after U minimum"
print(CurrentLabel)
print(OA_mean_pos_u_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

CurrentLabel<-"Proportion of average downward change"
prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
}
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "Average upward change after U minimum"
## [1] 0.009616732

if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "differences from left max to min for each row: "
## [1] 0.01999982 0.03706263 0.05793915 0.08025712 0.10055452 0.11903339 0.13945546
## [1] "differences from min to right max for each row: "
## [1] 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 0.004070529 0.019518221
## [1] " "
## [1] "row with biggest return upward"
## [1] 7
## [1] " "
## [1] "downward distance for row with the largest upward value"
## [1] 0.1394555
## [1] 0.01951822
## [1] " "
## [1] "Return upward as a proportion of the downward distance:"
## [1] 0.1399603

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",

```



```

## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos      stim
##      4.752292      -0.198252      0.362637      0.027558      -0.401710
##      log_freq:pos
##      0.078950
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4199 Residual
## Null Deviance:      2806
## Residual Deviance: 2651 AIC: 2844
## log likelihood: -1325.474
## Nagelkerke R2:  0.07445135
## % pres/err predicted correctly: -777.6119
## % of predictable range [ (model-null)/(1-null) ]:  0.03913396
## *****
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      log_freq
##      4.80094      -0.19086      0.03095      -0.43959      0.09411
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4202 Residual
## Null Deviance:      2806
## Residual Deviance: 2662 AIC: 2848
## log likelihood: -1330.788
## Nagelkerke R2:  0.06944316
## % pres/err predicted correctly: -779.934
## % of predictable range [ (model-null)/(1-null) ]:  0.03626828
## *****
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      log_freq      I(pos
##      4.739078      -0.190185      0.026931      -0.404818      -0.069168
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4200 Residual
## Null Deviance:      2806
## Residual Deviance: 2659 AIC: 2849
## log likelihood: -1329.372
## Nagelkerke R2:  0.07077931
## % pres/err predicted correctly: -779.1722
## % of predictable range [ (model-null)/(1-null) ]:  0.03720843
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```

##      (Intercept)          stimlen      log_freq          pos  stimlen:log_freq
##      4.13534          -0.17525      0.52522          -0.15377      -0.05214
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4202 Residual
## Null Deviance:      2806
## Residual Deviance: 2662 AIC: 2851
## log likelihood: -1331.065
## Nagelkerke R2:  0.06918198
## % pres/err predicted correctly: -779.6932
## % of predictable range [ (model-null)/(1-null) ]:  0.03656552
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)          stimlen      log_freq          pos  stimlen:log_freq
##      4.13613          -0.17923      0.53015          -0.14669      -0.06043
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4201 Residual
## Null Deviance:      2806
## Residual Deviance: 2661 AIC: 2852
## log likelihood: -1330.517
## Nagelkerke R2:  0.06969894
## % pres/err predicted correctly: -779.4081
## % of predictable range [ (model-null)/(1-null) ]:  0.03691726
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos      log_freq
##      4.12542      -0.17072      -0.15399      0.09513
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4203 Residual
## Null Deviance:      2806
## Residual Deviance: 2671 AIC: 2858
## log likelihood: -1335.301
## Nagelkerke R2:  0.06518008
## % pres/err predicted correctly: -781.2842
## % of predictable range [ (model-null)/(1-null) ]:  0.03460205
## *****
## model index:  20
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      5.01285      -0.21958      0.03131      -0.44263
##

```

```

## Degrees of Freedom: 4206 Total (i.e. Null); 4203 Residual
## Null Deviance: 2806
## Residual Deviance: 2673 AIC: 2858
## log likelihood: -1336.702
## Nagelkerke R2: 0.06385472
## % pres/err predicted correctly: -782.3133
## % of predictable range [ (model-null)/(1-null) ]: 0.03333209
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos stimlen:I(pos^2) stimlen:I(pos^2)
## 3.01491 0.01179 -0.12464 0.80620 0.01733 -0.01733
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4201 Residual
## Null Deviance: 2806
## Residual Deviance: 2669 AIC: 2858
## log likelihood: -1334.405
## Nagelkerke R2: 0.06602675
## % pres/err predicted correctly: -781.1732
## % of predictable range [ (model-null)/(1-null) ]: 0.0347391
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 4.123494 -0.169724 -0.155544 0.107363 -0.002658
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4202 Residual
## Null Deviance: 2806
## Residual Deviance: 2671 AIC: 2860
## log likelihood: -1335.276
## Nagelkerke R2: 0.06520349
## % pres/err predicted correctly: -781.2929
## % of predictable range [ (model-null)/(1-null) ]: 0.03459136
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos stimlen:pos
## 5.18426 -0.29899 -0.39068 0.02698
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4203 Residual
## Null Deviance: 2806
## Residual Deviance: 2680 AIC: 2866

```

```

## log likelihood: -1340.082
## Nagelkerke R2: 0.06065364
## % pres/err predicted correctly: -783.5009
## % of predictable range [ (model-null)/(1-null) ]: 0.03186653
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##      4.3317      -0.1996      -0.1534
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4204 Residual
## Null Deviance:      2806
## Residual Deviance: 2683 AIC: 2868
## log likelihood: -1341.346
## Nagelkerke R2: 0.05945542
## % pres/err predicted correctly: -783.6142
## % of predictable range [ (model-null)/(1-null) ]: 0.03172665
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      I(pos^2)          pos      log_freq I(pos^2):log_freq
##      3.306980      0.016936      -0.369829      -0.004650      -0.009644
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4201 Residual
## Null Deviance:      2806
## Residual Deviance: 2684 AIC: 2877
## log likelihood: -1342.071
## Nagelkerke R2: 0.05876772
## % pres/err predicted correctly: -783.9855
## % of predictable range [ (model-null)/(1-null) ]: 0.03126845
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##      2.9840      -0.2036      0.1250
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4204 Residual
## Null Deviance:      2806
## Residual Deviance: 2692 AIC: 2881
## log likelihood: -1346.126
## Nagelkerke R2: 0.05491705
## % pres/err predicted correctly: -785.383

```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.02954382
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos log_freq pos:log_freq
## 3.002197 -0.208409 0.169118 -0.009713
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4203 Residual
## Null Deviance: 2806
## Residual Deviance: 2692 AIC: 2882
## log likelihood: -1345.799
## Nagelkerke R2: 0.05522789
## % pres/err predicted correctly: -785.3109
## % of predictable range [ (model-null)/(1-null) ]: 0.02963284
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq stimlen:log_freq
## 4.16092 -0.25984 0.52373 -0.05204
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4203 Residual
## Null Deviance: 2806
## Residual Deviance: 2703 AIC: 2887
## log likelihood: -1351.564
## Nagelkerke R2: 0.04974148
## % pres/err predicted correctly: -789.2024
## % of predictable range [ (model-null)/(1-null) ]: 0.02483043
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq
## 4.15052 -0.25539 0.09397
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4204 Residual
## Null Deviance: 2806
## Residual Deviance: 2712 AIC: 2894
## log likelihood: -1355.829
## Nagelkerke R2: 0.04567313
## % pres/err predicted correctly: -790.7697
## % of predictable range [ (model-null)/(1-null) ]: 0.02289628
## *****
## model index: 19

```

```

##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      3.36526      0.02025     -0.40714
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4204 Residual
## Null Deviance:      2806
## Residual Deviance: 2710  AIC: 2898
## log likelihood:  -1355.226
## Nagelkerke R2:  0.04624884
## % pres/err predicted correctly:  -789.881
## % of predictable range [ (model-null)/(1-null) ]:  0.02399298
## *****
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.0116      -0.2164
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2714  AIC: 2902
## log likelihood:  -1357.219
## Nagelkerke R2:  0.044345
## % pres/err predicted correctly:  -790.0866
## % of predictable range [ (model-null)/(1-null) ]:  0.02373923
## *****
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.3544      -0.2837
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2724  AIC: 2904
## log likelihood:  -1361.802
## Nagelkerke R2:  0.03996133
## % pres/err predicted correctly:  -792.8356
## % of predictable range [ (model-null)/(1-null) ]:  0.02034676
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)

```

```
##
## Coefficients:
## (Intercept)
##      2.1
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4206 Residual
## Null Deviance:      2806
## Residual Deviance: 2806  AIC: 2990
## log likelihood:  -1403.123
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -809.323
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                           AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	log_freq	log_freq + I(pos^2)	log_freq + I(pos^2) + pos	log_freq + I(pos^2) + pos^2	log_freq + I(pos^2) + pos^3	log_freq + I(pos^2) + pos^4	log_freq + I(pos^2) + pos^5
preserved ~ stimlen *	2842.389	0.0000000000000000	0.9985578	0.875	0.258	0.5117527	-	NA	NA	0.0296966	NA	NA	NA
log_freq + I(pos^2) + pos						0.1947626	0.0505756	0.276635					
preserved ~ stimlen *	2843.536	1.147	0.0000000000000000	0.363	0.379	0.4752292	0.3626366	-	NA	0.0789108	0.7576	-	NA
log_freq + (I(pos^2) + pos) *						0.1982515	0.0555660	0.17096			0.0067352		
log_freq													
preserved ~ stimlen + I(pos^2) + pos + log_freq	2848.178	5.790	0.0000000000000000	0.330	0.169	0.4890944	0.0941040	-	NA	NA	0.0309461	NA	NA
						0.1908597		0.4395906					

[illegible]

Model	AIC Delta	AIC	AICw	NagR ²	(Intercept)	stimlen	log_freq	log_pos	log_freq:I(pos)	I(pos^2)	log_freq:I(pos)	pos	stimlen:log_freq	I(pos^2)
preserved ~ I(pos^2) + pos	2898.565	5807.000	0.0000000	0.6348526	NA	NA	-	NA	NA	0.0202477	NA	NA	NA	NA
preserved ~ pos	2901.598	2070.000	0.0000000	0.4501611	NA	NA	-	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen	2903.692	2092.000	0.0000000	0.3943541	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ 1	2989.971	5813.000	0.0000000	0.099764	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen * log_freq + I(pos^2) + pos"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          stimlen          log_freq          I(pos^2)          pos  stimlen:log_freq
## 4.78526      -0.19476          0.51175          0.02970      -0.42766          -0.00000
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4201 Residual
## Null Deviance: 2806
## Residual Deviance: 2654 AIC: 2842
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

```
HF_Plot <- plot_len_pos_obs_predicted(HFDat, paste0(CurPat, " - ", RunLabel, " - High frequency"), "FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
LF_Plot <- plot_len_pos_obs_predicted(LFDat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted")
```

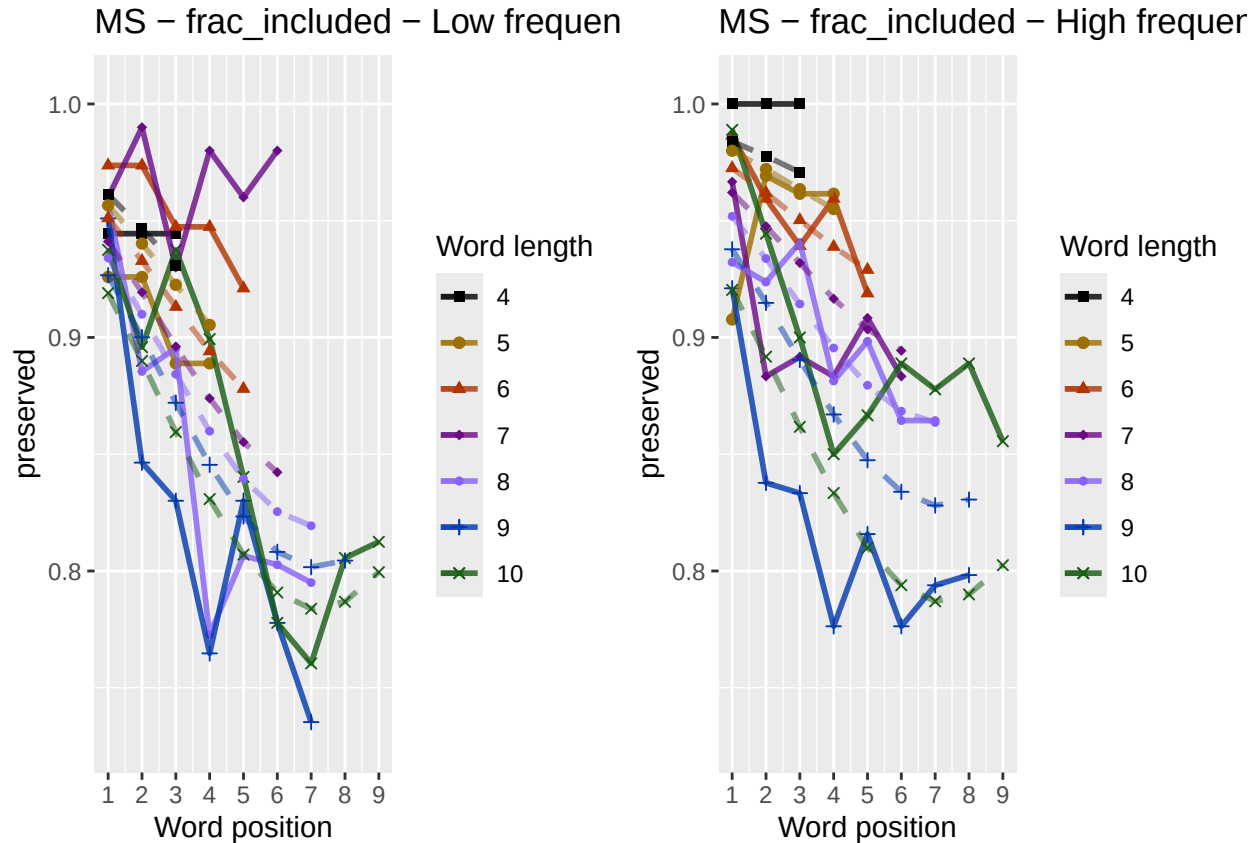
```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
library(ggpubr)
```

```
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
              CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)
```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## *****
## model index: 2
##
```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.967      -1.627
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 1870 AIC: 1982
## log likelihood: -935.193
## Nagelkerke R2:  0.4097315
## % pres/err predicted correctly: -508.9251
## % of predictable range [ (model-null)/(1-null) ]:  0.3707138
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      3.36526      0.02025     -0.40714
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4204 Residual
## Null Deviance:      2806
## Residual Deviance: 2710 AIC: 2898
## log likelihood: -1355.226
## Nagelkerke R2:  0.04624884
## % pres/err predicted correctly: -789.881
## % of predictable range [ (model-null)/(1-null) ]:  0.02399298
## *****
## model index:  4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.0116     -0.2164
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2714 AIC: 2902
## log likelihood: -1357.219
## Nagelkerke R2:  0.044345
## % pres/err predicted correctly: -790.0866
## % of predictable range [ (model-null)/(1-null) ]:  0.02373923
## *****
## model index:  5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##

```

```

## Coefficients:
## (Intercept)      stimlen
##      4.3544      -0.2837
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2724  AIC: 2904
## log likelihood:  -1361.802
## Nagelkerke R2:  0.03996133
## % pres/err predicted correctly:  -792.8356
## % of predictable range [ (model-null)/(1-null) ]:  0.02034676
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.7315      0.1603
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2768  AIC: 2948
## log likelihood:  -1384.235
## Nagelkerke R2:  0.0183633
## % pres/err predicted correctly:  -802.3399
## % of predictable range [ (model-null)/(1-null) ]:  0.008617722
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.1
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4206 Residual
## Null Deviance:      2806
## Residual Deviance: 2806  AIC: 2990
## log likelihood:  -1403.123
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -809.323
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****

BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)

```

```

MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1981.610	0.0000	1	1	0.4097312	1.966741	NA	-	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2898.463	16.8554	0	0	0.0462488	3.365260	NA	NA	0.0202477	-	NA
preserved ~ pos	2901.708	20.0984	0	0	0.0443450	3.011649	NA	NA	NA	-	NA
preserved ~ stimlen	2903.792	22.1829	0	0	0.0399613	3.354417	NA	NA	NA	NA	-
preserved ~ CumPres	2948.279	66.6691	0	0	0.0183633	3.731486	0.1602528	NA	NA	NA	NA
preserved ~ 1	2989.971	1008.3618	0	0	0.0000000	0.099764	NA	NA	NA	NA	NA

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
  BestMEModelRndDF <- rbind(BestMEModelRndDF,
                            data.frame(Name=c("Random SD"),

```

```

AIC=c(sd(RndModelAIC)))

write.csv(BestMEModelRndDF,
  paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
    "_best_main_effects_model_with_random_cum_term.csv"),
  row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
    N = n())
write.csv(syll_component_summary, paste0(TablesDir, CurPat, "_",
  RunLabel, "_", CurTask,
  "_syllable_component_summary.csv"), row.names = FALSE)

kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.9135135	555
O	0.8876308	1943
P	0.9333333	30
S	0.8833333	240
V	0.8869122	1439

```

# main effects models for data without satellite positions

keep_components = c("0", "V", "1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
  stimlen, stim, pos,
  preserved, syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data, MEModelEquations)

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.946      -1.694
##

```

```

## Degrees of Freedom: 3936 Total (i.e. Null); 3935 Residual
## Null Deviance: 2623
## Residual Deviance: 1765 AIC: 1882
## log likelihood: -882.6009
## Nagelkerke R2: 0.4025918
## % pres/err predicted correctly: -480.7765
## % of predictable range [ (model-null)/(1-null) ]: 0.363935
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 3.43298 0.02053 -0.42168
##
## Degrees of Freedom: 3936 Total (i.e. Null); 3934 Residual
## Null Deviance: 2623
## Residual Deviance: 2525 AIC: 2705
## log likelihood: -1262.422
## Nagelkerke R2: 0.05071851
## % pres/err predicted correctly: -736.5238
## % of predictable range [ (model-null)/(1-null) ]: 0.02628498
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 3.0676 -0.2269
##
## Degrees of Freedom: 3936 Total (i.e. Null); 3935 Residual
## Null Deviance: 2623
## Residual Deviance: 2529 AIC: 2708
## log likelihood: -1264.349
## Nagelkerke R2: 0.04875387
## % pres/err predicted correctly: -736.6482
## % of predictable range [ (model-null)/(1-null) ]: 0.02612075
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 4.4318 -0.2928
##
## Degrees of Freedom: 3936 Total (i.e. Null); 3935 Residual
## Null Deviance: 2623
## Residual Deviance: 2541 AIC: 2712

```

```
## log likelihood: -1270.638
## Nagelkerke R2: 0.04233112
## % pres/err predicted correctly: -740.1801
## % of predictable range [ (model-null)/(1-null) ]: 0.02145781
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 1.7747 0.1505
##
## Degrees of Freedom: 3936 Total (i.e. Null); 3935 Residual
## Null Deviance: 2623
## Residual Deviance: 2595 AIC: 2766
## log likelihood: -1297.307
## Nagelkerke R2: 0.01486489
## % pres/err predicted correctly: -751.1418
## % of predictable range [ (model-null)/(1-null) ]: 0.006985619
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.101
##
## Degrees of Freedom: 3936 Total (i.e. Null); 3936 Residual
## Null Deviance: 2623
## Residual Deviance: 2623 AIC: 2797
## log likelihood: -1311.591
## Nagelkerke R2: 2.282581e-16
## % pres/err predicted correctly: -756.4329
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1881.6190	0.0000	1	1	0.4025918	9.946495	NA	-	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2705.0228	23.4031	0	0	0.0507183	3.432984	NA	NA	0.020533	-	NA
preserved ~ pos	2707.8078	26.1879	0	0	0.0487533	3.067559	NA	NA	NA	-	NA
										0.4216755	
										0.2268898	

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ stimlen	2712.297	30.6779	0	0	0.0423314	1.431803	NA	NA	NA	NA	-
preserved ~ CumPres	2766.418	84.7992	0	0	0.0148649	1.774730	0.150492	NA	NA	NA	0.2927751
preserved ~ 1	2797.447	115.8277	0	0	0.0000000	1.101170	NA	NA	NA	NA	NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.840      -1.778
##
## Degrees of Freedom: 3381 Total (i.e. Null); 3380 Residual
## Null Deviance:      2301
## Residual Deviance: 1621 AIC: 1735
## log likelihood: -810.2935
## Nagelkerke R2: 0.3693177
## % pres/err predicted correctly: -446.3918
## % of predictable range [ (model-null)/(1-null) ]: 0.3317669
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```

## (Intercept)      I(pos^2)          pos
##      3.41758      0.02501      -0.45300
##
## Degrees of Freedom: 3381 Total (i.e. Null); 3379 Residual
## Null Deviance:      2301
## Residual Deviance: 2214 AIC: 2373
## log likelihood: -1107.235
## Nagelkerke R2: 0.05131252
## % pres/err predicted correctly: -650.9246
## % of predictable range [ (model-null)/(1-null) ]: 0.02627266
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      2.9871      -0.2178
##
## Degrees of Freedom: 3381 Total (i.e. Null); 3380 Residual
## Null Deviance:      2301
## Residual Deviance: 2220 AIC: 2378
## log likelihood: -1109.956
## Nagelkerke R2: 0.0481324
## % pres/err predicted correctly: -651.3378
## % of predictable range [ (model-null)/(1-null) ]: 0.02565553
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.358      -0.289
##
## Degrees of Freedom: 3381 Total (i.e. Null); 3380 Residual
## Null Deviance:      2301
## Residual Deviance: 2230 AIC: 2379
## log likelihood: -1114.799
## Nagelkerke R2: 0.04246011
## % pres/err predicted correctly: -653.7097
## % of predictable range [ (model-null)/(1-null) ]: 0.02211284
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      1.88210      0.09645
##

```

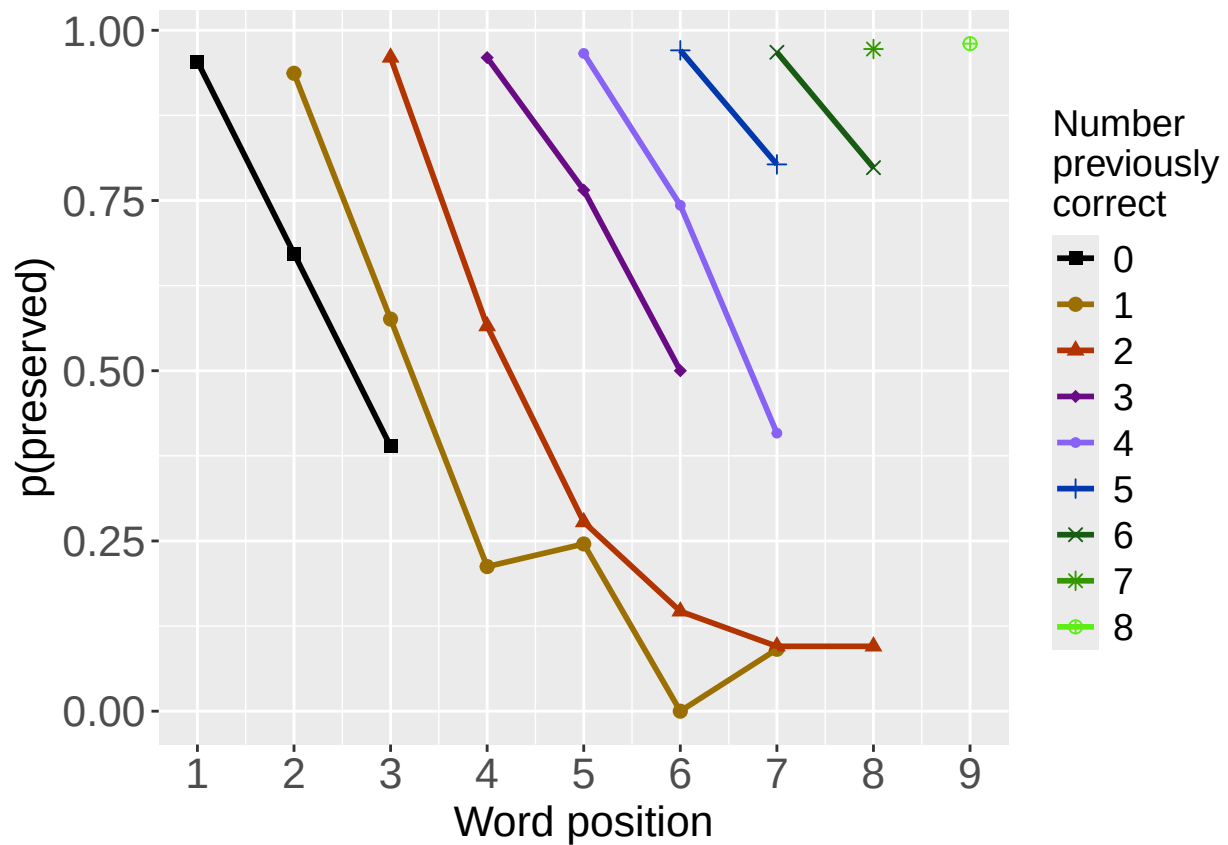
```
## Degrees of Freedom: 3381 Total (i.e. Null); 3380 Residual
## Null Deviance: 2301
## Residual Deviance: 2293 AIC: 2445
## log likelihood: -1146.593
## Nagelkerke R2: 0.004814679
## % pres/err predicted correctly: -666.9288
## % of predictable range [ (model-null)/(1-null) ]: 0.002368535
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.064
##
## Degrees of Freedom: 3381 Total (i.e. Null); 3381 Residual
## Null Deviance: 2301
## Residual Deviance: 2301 AIC: 2453
## log likelihood: -1150.617
## Nagelkerke R2: 2.24922e-16
## % pres/err predicted correctly: -668.5145
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1734.7510	0.0000	1	1	0.3693172	8.39991	NA	-	NA	NA	NA
preserved ~ (I(pos^2) + pos)	2372.9016	38.1502	0	0	0.0513123	3.417579	NA	NA	0.025008	-	NA
preserved ~ pos	2377.7576	43.0062	0	0	0.0481322	2.987127	NA	NA	NA	-	NA
preserved ~ stimlen	2378.9196	44.1681	0	0	0.0424601	1.357984	NA	NA	NA	NA	-
preserved ~ CumPres	2444.9497	10.1982	0	0	0.0048141	1.882100	0.096449	NA	NA	NA	NA
preserved ~ 1	2453.3047	18.5530	0	0	0.0000000	0.063704	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

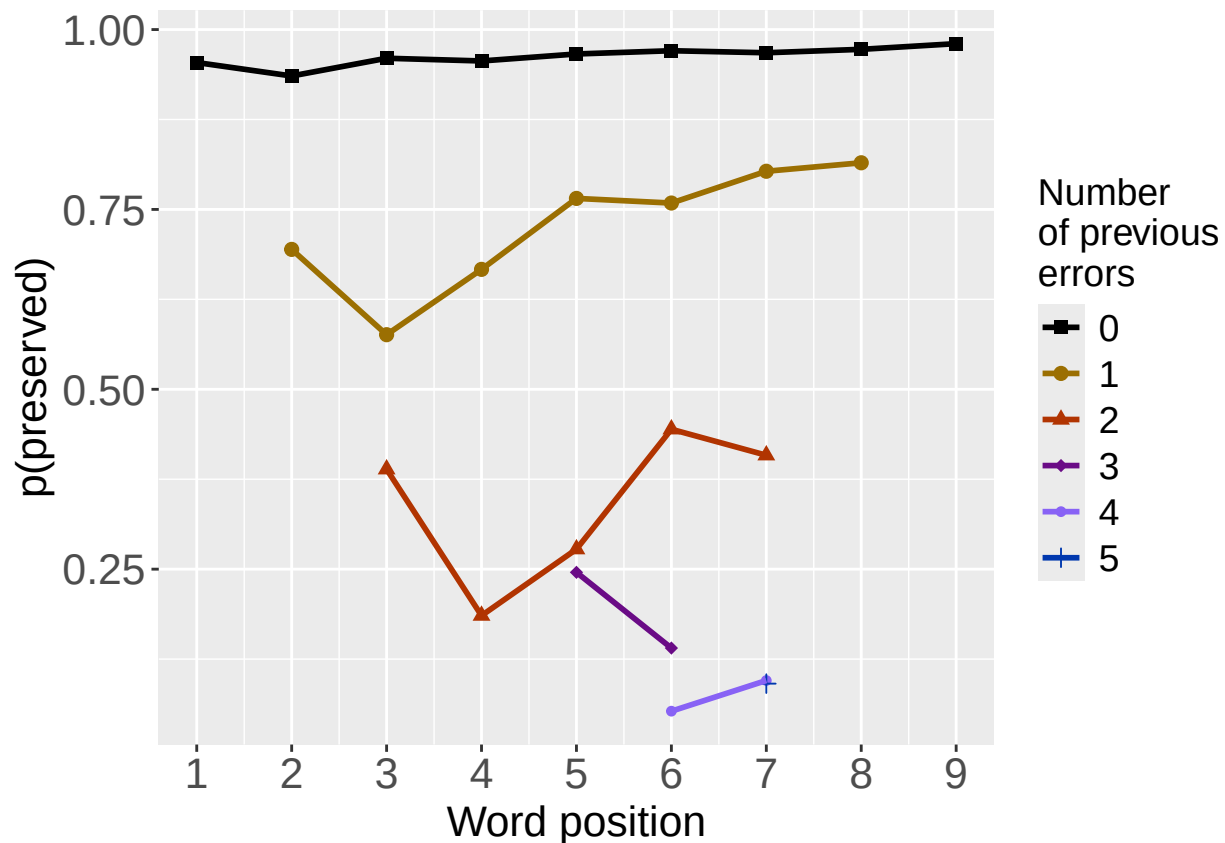
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

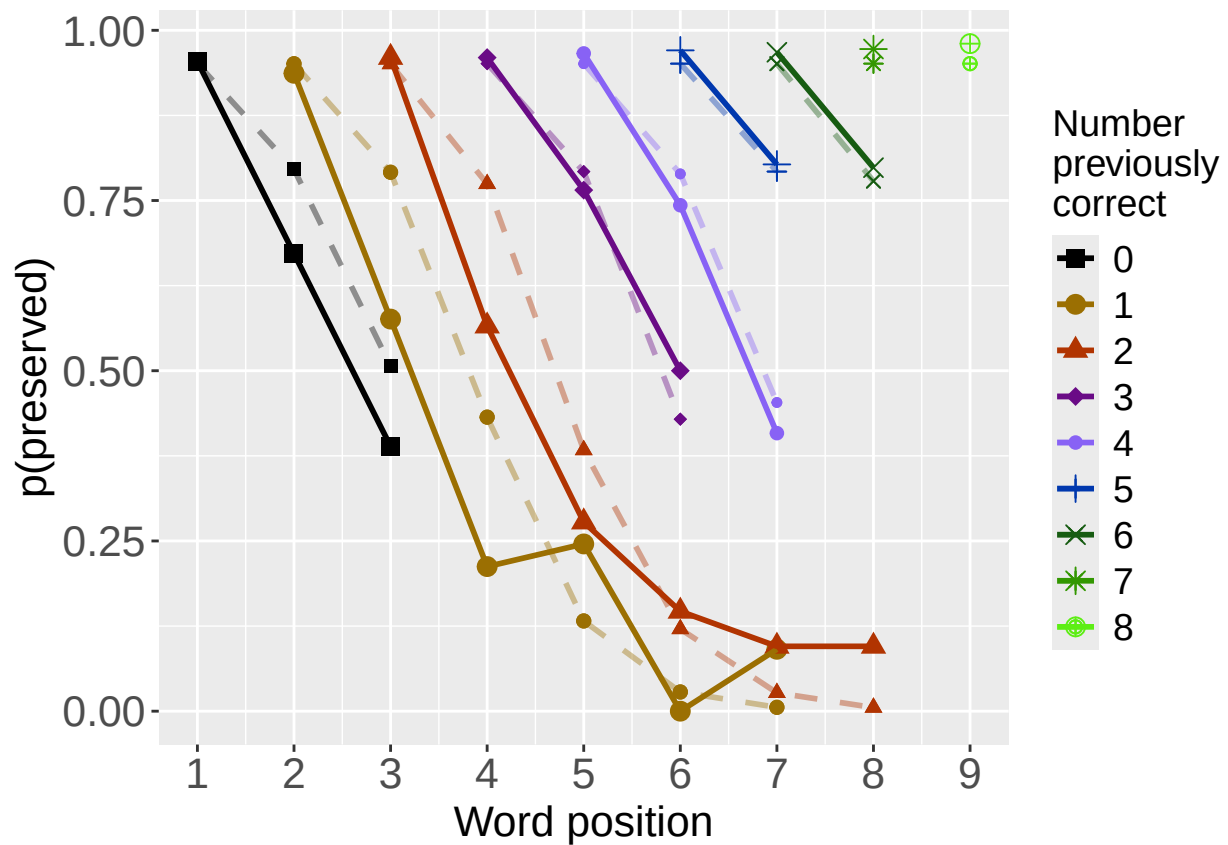
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

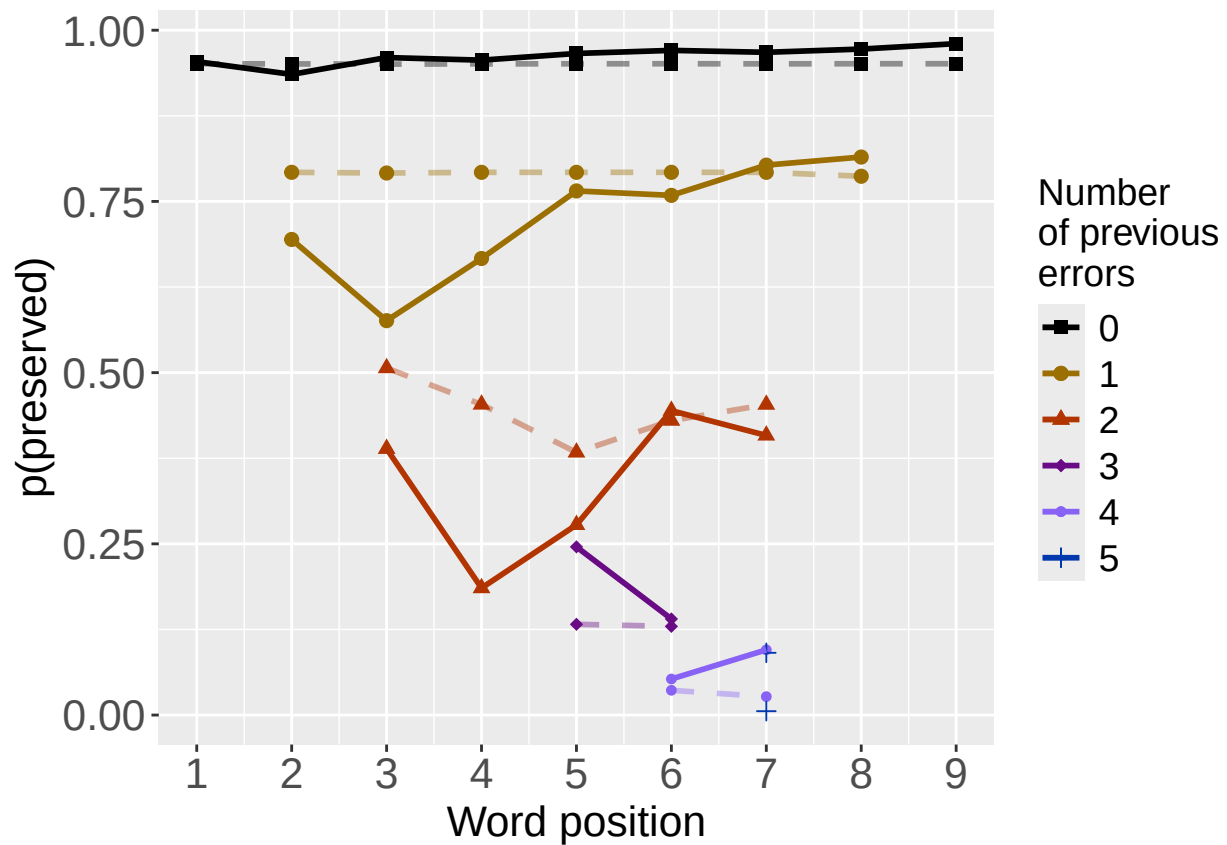
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

Saving 6.5 x 4.5 in image

```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

Saving 6.5 x 4.5 in image

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## *****
## model index: 2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##      3.1250      -1.7776      0.0471      -0.2542
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4203 Residual
## Null Deviance:      2806
## Residual Deviance: 1840  AIC: 1947
## log likelihood:  -920.1705
## Nagelkerke R2:  0.421435
## % pres/err predicted correctly:  -500.4278
## % of predictable range [ (model-null)/(1-null) ]:  0.3812002

```

```

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      2.967      -1.627
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 1870 AIC: 1982
## log likelihood:  -935.193
## Nagelkerke R2:  0.4097315
## % pres/err predicted correctly:  -508.9251
## % of predictable range [ (model-null)/(1-null) ]:  0.3707138
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      3.36526      0.02025     -0.40714
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4204 Residual
## Null Deviance:      2806
## Residual Deviance: 2710 AIC: 2898
## log likelihood:  -1355.226
## Nagelkerke R2:  0.04624884
## % pres/err predicted correctly:  -789.881
## % of predictable range [ (model-null)/(1-null) ]:  0.02399298
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1946.609	0.00000	1	1	0.4214350	3.124989	-1.777583	0.0470997	-0.2542320

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr	1981.610	35.00086	0	0	0.4097315	2.966741	-1.626647	NA	NA
preserved ~ I(pos^2) + pos	2898.465	951.85623	0	0	0.0462488	3.365260	NA	0.0202477	-0.4071437

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 3.7695 -1.5884 -0.1044
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4204 Residual
## Null Deviance: 2806
## Residual Deviance: 1863 AIC: 1976
## log likelihood: -931.6187
## Nagelkerke R2: 0.4125238
## % pres/err predicted correctly: -508.3409
## % of predictable range [ (model-null)/(1-null) ]: 0.3714348
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.967 -1.627
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4205 Residual
## Null Deviance: 2806
## Residual Deviance: 1870 AIC: 1982
## log likelihood: -935.193
## Nagelkerke R2: 0.4097315
## % pres/err predicted correctly: -508.9251
## % of predictable range [ (model-null)/(1-null) ]: 0.3707138
## *****
## model index: 3
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      4.3544      -0.2837
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2724 AIC: 2904
## log likelihood: -1361.802
## Nagelkerke R2:  0.03996133
## % pres/err predicted correctly: -792.8356
## % of predictable range [ (model-null)/(1-null) ]:  0.02034676
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	1975.916	0.000000	1.0000000	0.9451531	0.4125238	3.769484	-	-
preserved ~ CumErr	1981.610	5.693603	0.0580296	0.0548469	0.4097315	2.966741	-	NA
preserved ~ stimlen	2903.792	927.876461	0.0000000	0.0000000	0.0399613	4.354417	NA	-
								0.2836826

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## *****
## model index:  2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.6292      -1.6338      0.1437
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4204 Residual
```

```

## Null Deviance:      2806
## Residual Deviance: 1852 AIC: 1959
## log likelihood:    -925.8664
## Nagelkerke R2:    0.4170073
## % pres/err predicted correctly: -504.0824
## % of predictable range [ (model-null)/(1-null) ]:  0.3766901
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##          2.967      -1.627
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 1870 AIC: 1982
## log likelihood:    -935.193
## Nagelkerke R2:    0.4097315
## % pres/err predicted correctly: -508.9251
## % of predictable range [ (model-null)/(1-null) ]:  0.3707138
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##          1.7315      0.1603
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2768 AIC: 2948
## log likelihood:    -1384.235
## Nagelkerke R2:    0.0183633
## % pres/err predicted correctly: -802.3399
## % of predictable range [ (model-null)/(1-null) ]:  0.008617722
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1958.857	0.00000	1.00e+00	0.9999885	0.4170073	2.629166	- 1.633760	0.1436841
preserved ~ CumErr	1981.610	22.75271	1.15e-05	0.0000115	0.4097315	2.966741	- 1.626647	NA
preserved ~ CumPres	2948.279	989.42179	0.00e+00	0.0000000	0.0183633	1.731486	NA	0.1602528

```

#####
# level 2 -- Add linear position (NOT quadratic)
#####

```

```

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr pos
## 2.4855 -1.7774 0.1437
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4204 Residual
## Null Deviance: 2806
## Residual Deviance: 1852 AIC: 1959
## log likelihood: -925.8664
## Nagelkerke R2: 0.4170073
## % pres/err predicted correctly: -504.0824
## % of predictable range [ (model-null)/(1-null) ]: 0.3766901
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 2.967 -1.627
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4205 Residual
## Null Deviance: 2806
## Residual Deviance: 1870 AIC: 1982
## log likelihood: -935.193
## Nagelkerke R2: 0.4097315
## % pres/err predicted correctly: -508.9251
## % of predictable range [ (model-null)/(1-null) ]: 0.3707138
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##

```

```
## Coefficients:
## (Intercept)          pos
##      3.0116      -0.2164
##
## Degrees of Freedom: 4206 Total (i.e. Null);  4205 Residual
## Null Deviance:      2806
## Residual Deviance: 2714  AIC: 2902
## log likelihood:  -1357.219
## Nagelkerke R2:  0.044345
## % pres/err predicted correctly:  -790.0866
## % of predictable range [ (model-null)/(1-null) ]:  0.02373923
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	1958.857	0.00000	1.00e+00	0.9999885	0.4170073	2.485482	-	0.1436841
+ pos							1.777445	
preserved ~ CumErr	1981.610	22.75271	1.15e-	0.0000115	0.4097315	2.966741	-	NA
			05				1.626647	
preserved ~ pos	2901.708	942.85113	0.00e+00	0.0000000	0.0443450	3.011649	NA	-
								0.2164271

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_main_effects_plus_one_model_summary.csv"),row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErrI(pos^2)	pos	stimlen	CumPres
preserved ~	1946.600	0.000000	1.000000	0.000000	0.421435	0.124989	-	0.0470997	-	NA
CumErr +							1.777583	0.2542320		
I(pos^2) + pos										
preserved ~	1958.850	0.000000	1.000000	0.9999885	0.4170073	2.629166	-	NA	NA	0.1436841
CumErr +							1.633760			
CumPres										
preserved ~	1958.850	0.000000	1.000000	0.9999885	0.4170073	2.485482	-	NA	0.1436841	NA
CumErr + pos							1.777445			
preserved ~	1975.910	0.000000	1.000000	0.0945153	0.4125238	3.769484	-	NA	NA	-
CumErr + stimlen							1.588359		0.1043556	
preserved ~	1981.610	0.000858	0.000000	0.000000	0.4097315	2.966741	-	NA	NA	NA
CumErr							1.626647			
preserved ~	1981.610	0.6936030	0.0580296	0.0548469	0.4097315	2.966741	-	NA	NA	NA
CumErr							1.626647			
preserved ~	1981.610	2.752710	0.000001	0.000001	0.4097315	2.966741	-	NA	NA	NA
CumErr							1.626647			
preserved ~	1981.610	2.752710	0.000001	0.000001	0.4097315	2.966741	-	NA	NA	NA
CumErr							1.626647			
preserved ~	2898.469	51.856207	0.000000	0.000000	0.0462488	3.365260	NA	0.0202477	-	NA
I(pos^2) + pos								0.4071437		
preserved ~ pos	2901.708	942.85113	0.000000	0.000000	0.0443450	3.011649	NA	NA	-	NA
								0.2164271		
preserved ~	2903.792	27.876461	0.000000	0.000000	0.0399614	3.354417	NA	NA	NA	-
stimlen									0.2836826	

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErrI	(pos^2)	pos	stimlen	CumPres
preserved ~ CumPres	2948.27989	4.421792	0.000000	0.000000	0.000000	0.18363	3731486	NA	NA	NA	0.1602528

```
# explore influence of frequency and length

if(grepl("stimlen",BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq")
  )
}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)          pos      stimlen      log_freq
##      4.4305      -1.7524      0.0571      -0.2837      -0.1737      0.1004
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4201 Residual
## Null Deviance:      2806
## Residual Deviance: 1808 AIC: 1915
## log likelihood: -903.8147
## Nagelkerke R2: 0.4340823
## % pres/err predicted correctly: -495.8026
```

```

## % of predictable range [ (model-null)/(1-null) ]: 0.3869079
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos      stimlen
##      4.64912      -1.75837      0.05658      -0.28250      -0.20404
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4202 Residual
## Null Deviance:      2806
## Residual Deviance: 1816 AIC: 1920
## log likelihood: -908.0139
## Nagelkerke R2: 0.4308446
## % pres/err predicted correctly: -496.3137
## % of predictable range [ (model-null)/(1-null) ]: 0.3862772
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos      log_freq
##      3.13052      -1.76628      0.04978      -0.26067      0.13555
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4202 Residual
## Null Deviance:      2806
## Residual Deviance: 1824 AIC: 1932
## log likelihood: -911.9783
## Nagelkerke R2: 0.427782
## % pres/err predicted correctly: -498.8503
## % of predictable range [ (model-null)/(1-null) ]: 0.3831468
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      pos
##      3.1250      -1.7776      0.0471      -0.2542
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4203 Residual
## Null Deviance:      2806
## Residual Deviance: 1840 AIC: 1947
## log likelihood: -920.1705
## Nagelkerke R2: 0.421435
## % pres/err predicted correctly: -500.4278
## % of predictable range [ (model-null)/(1-null) ]: 0.3812002
## *****
## model index: 2

```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 2.1
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4206 Residual
## Null Deviance: 2806
## Residual Deviance: 2806 AIC: 2990
## log likelihood: -1403.123
## Nagelkerke R2: 0
## % pres/err predicted correctly: -809.323
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	log_freq	stimlen
preserved ~ CumErr + I(pos^2) + pos + stimlen + log_freq	1914.656	0.000000	1.000000	0.450052	0.340823	130505	-	0.0571045	-	0.1004041	-
							1.752372		0.2837288		0.1736678
preserved ~ CumErr + I(pos^2) + pos + stimlen	1920.349	6929580	0.058048	0.548560	0.130844	649117	-	0.0565806	-	NA	-
							1.758366		0.2825008		0.2040398
preserved ~ CumErr + I(pos^2) + pos + log_freq	1932.307	6765290	0.000140	0.000138	0.727783	130520	-	0.0497778	-	0.1355503	NA
							1.766285		0.2606694		
preserved ~ CumErr + I(pos^2) + pos	1946.609	1952880	0.000000	0.000000	0.421433	124989	-	0.0470997	-	NA	NA
							1.777583		0.2542320		
preserved ~ 1	2989.971	1075.315	0.000000	0.000000	0.000000	0099764	NA	NA	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
```

```

# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

## Single term deletions
##
## Model:
## preserved ~ CumErr + I(pos^2) + pos + stimlen + log_freq
##           Df Deviance   AIC
## CumErr    1   2661.6 2766.6
## I(pos^2)   1   1824.0 1929.1
## stimlen    1   1824.0 1929.0
## log_freq   1   1816.0 1921.0
## pos        1   1812.8 1917.8
## <none>      1   1807.6 1914.7

#####
# Single deletions from best model
#####

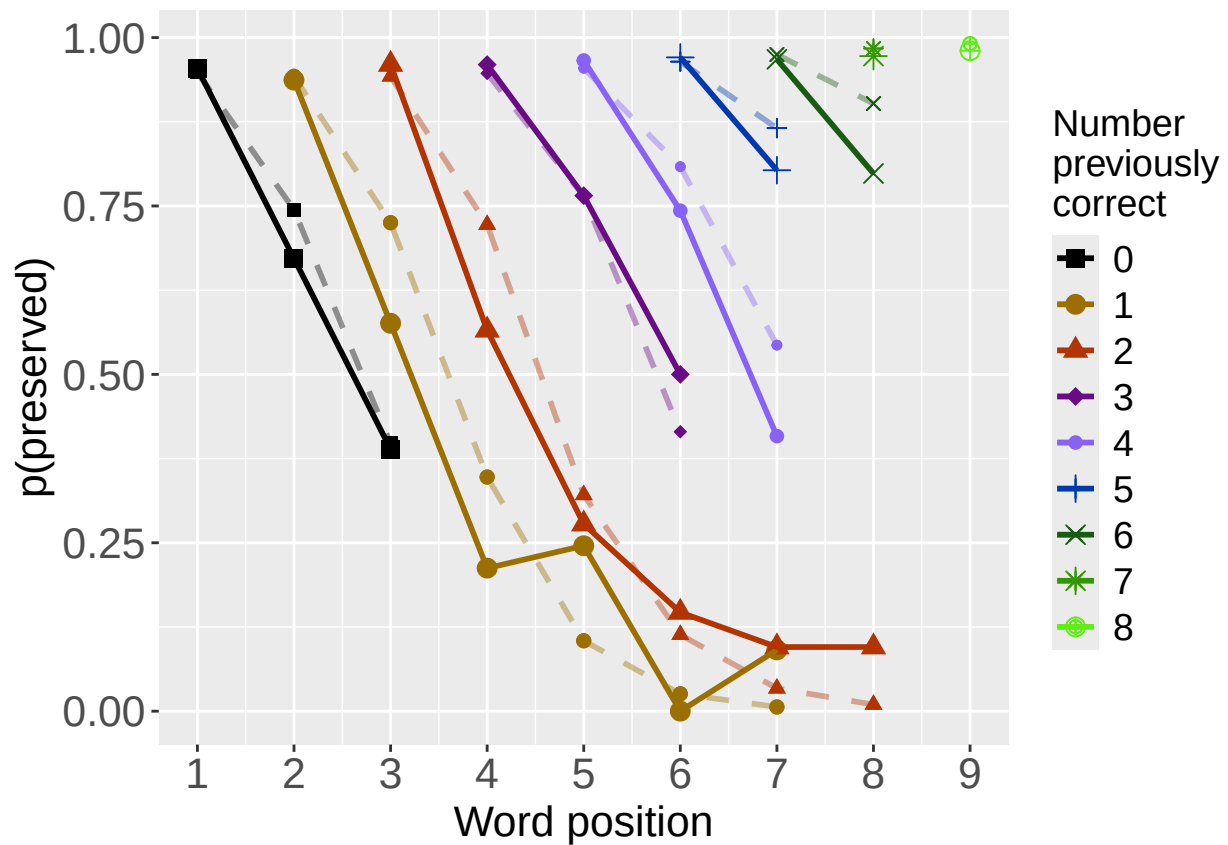
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

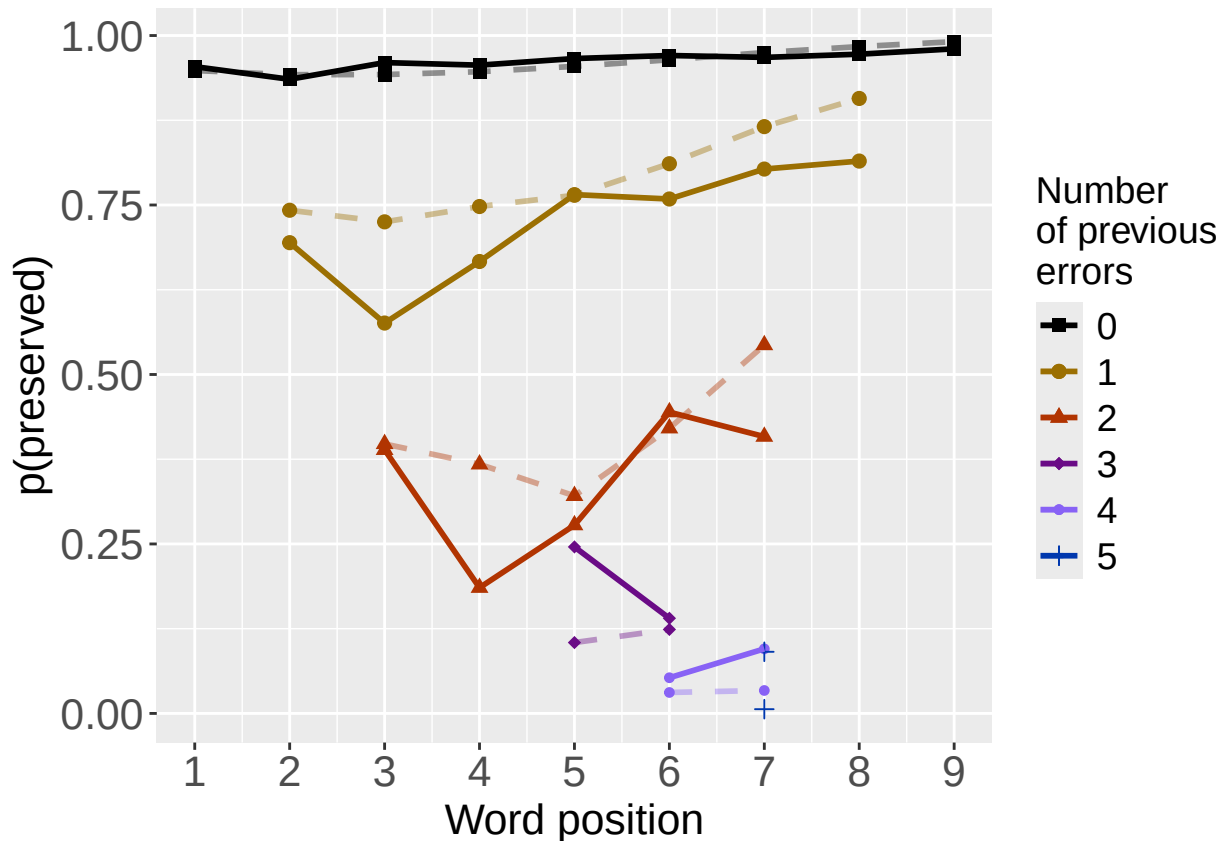
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
  RunLabel,"_",CurTask,
  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
      family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

```

```

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

```

```

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

```

```

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

```

```

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

```

```

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

```

```

## *****

```

```

## model index: 1
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)

```

```

##

```

```

## Coefficients:

```

```

## (Intercept)      CumErr
##      2.967      -1.627
##

```

```

## Degrees of Freedom: 4206 Total (i.e. Null); 4205 Residual

```

```

## Null Deviance:      2806

```

```

## Residual Deviance: 1870  AIC: 1982
## log likelihood: -935.193
## Nagelkerke R2: 0.4097315
## % pres/err predicted correctly: -508.9251
## % of predictable range [ (model-null)/(1-null) ]: 0.3707138
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)
##      2.68170      -1.79851      0.01939
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4204 Residual
## Null Deviance:      2806
## Residual Deviance: 1845  AIC: 1950
## log likelihood: -922.2764
## Nagelkerke R2: 0.4197994
## % pres/err predicted correctly: -501.9392
## % of predictable range [ (model-null)/(1-null) ]: 0.3793349
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      stimlen
##      4.11600      -1.78198      0.02566      -0.19847
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4203 Residual
## Null Deviance:      2806
## Residual Deviance: 1821  AIC: 1925
## log likelihood: -910.6033
## Nagelkerke R2: 0.4288449
## % pres/err predicted correctly: -498.0372
## % of predictable range [ (model-null)/(1-null) ]: 0.3841502
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)      stimlen      log_freq
##      3.89604      -1.77633      0.02601      -0.16814      0.10010
##
## Degrees of Freedom: 4206 Total (i.e. Null); 4202 Residual
## Null Deviance:      2806
## Residual Deviance: 1813  AIC: 1919
## log likelihood: -906.4136
## Nagelkerke R2: 0.4320792

```

```

## % pres/err predicted correctly: -497.5917
## % of predictable range [ (model-null)/(1-null) ]: 0.3847001
## *****

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

```

```
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).

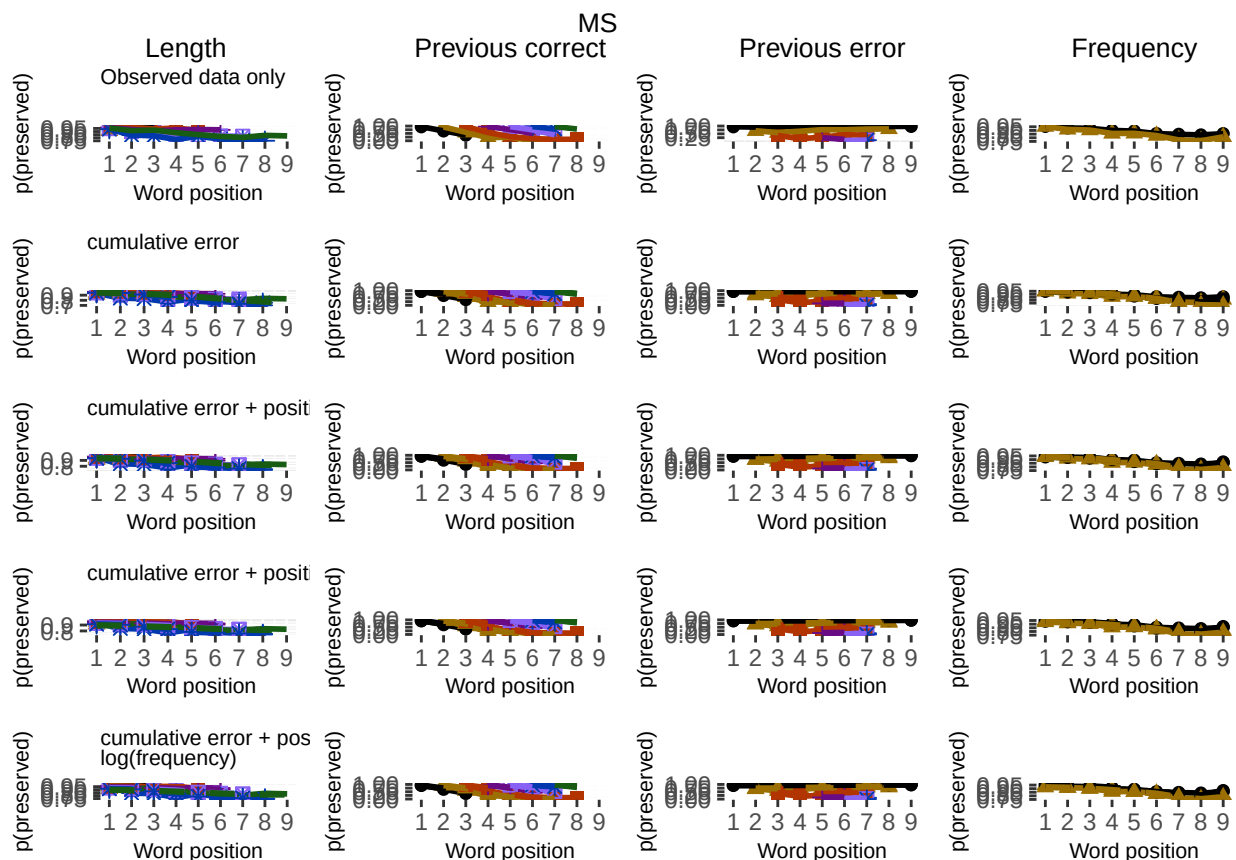
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).

ggsave(paste0(PlotName,".tif"),plot=FactorPlot,width = 360,height=400,units="mm",device="tiff",compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
```

```
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
```

[illegible]

```
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
```

```
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                       CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	I(pos^2)	pos	stimlen	log_freq
McFadden	0.3205489	0.0119448	0.0121646	0.0129050	0.0056649
SquaredCorrelation	0.1992640	0.0076897	0.0081402	0.0085679	0.0037264
Nagelkerke	0.1992640	0.0076897	0.0081402	0.0085679	0.0037264
Estrella	0.2426778	0.0089527	0.0089223	0.0095089	0.0041970

	deviance	deviance_explained
CumErr + I(pos^2) + stimlen + log_freq	1812.827	993.4181
CumErr + I(pos^2) + stimlen	1821.207	985.0388
CumErr + I(pos^2)	1844.553	961.6926
CumErr	1870.386	935.8593
null	2806.245	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
```

```
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
## Warning in eval(family$initialize): non-integer #successes in a binomial glm!
```

```
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
```

```
        row.names = FALSE)
```

```
deviance_differences_df
```

```
##
## CumErr + I(pos^2) + stimlen + log_freq CumErr + I(pos^2) + stimlen + log_freq 1812.827      993.4181      35.40026
## CumErr + I(pos^2) + stimlen          CumErr + I(pos^2) + stimlen 1821.207      985.0388      35.10166
## CumErr + I(pos^2)                  CumErr + I(pos^2) 1844.553      961.6926      34.26973
## CumErr                          CumErr 1870.386      935.8593      33.34916
## null                            null 2806.245        0.0000      0.00000
```

```
##
## percent_of_explained_deviance increment_in_explained
## CumErr + I(pos^2) + stimlen + log_freq 100.00000      0.8434836
## CumErr + I(pos^2) + stimlen          99.15652      2.3500855
## CumErr + I(pos^2)                  96.80643      2.6004482
## CumErr                          94.20598      94.2059828
## null                            NA      0.0000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + I(pos ²) + stimlen + log_freq	35.40026	100.00000	0.8434836
CumErr + I(pos ²) + stimlen	35.10166	99.15652	2.3500855
CumErr + I(pos ²)	34.26973	96.80643	2.6004482
CumErr	33.34916	94.20598	94.2059828
null	0.00000	NA	0.0000000

```

NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##          Nagelkerke
## CumErr    0.87631638
## I(pos^2)  0.03381750
## pos       0.03579861
## stimlen   0.03767975
## log_freq  0.01638777

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in eval(family$initialize): non-integer #successes in a binomial glm!

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

```

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.9744199	1870.386
preserved ~ CumErr+I(pos^2)	0.9839958	1844.553
preserved ~ CumErr+I(pos^2)+stimlen+log_freq	0.9842432	1812.827
preserved ~ CumErr+I(pos^2)+stimlen	0.9843352	1821.207

model	diff_CumErr	diff_CumErr+I(pos^2)	diff_CumErr+I(pos^2)+stimlen+log_freq	diff_CumErr+I(pos^2)+stimlen
preserved ~ CumErr	0.0000000	-0.0095759	-0.0098233	-0.0099153
preserved ~ CumErr+I(pos^2)	0.0095759	0.0000000	-0.0002474	-0.0003394
preserved ~ CumErr+I(pos^2)+stimlen+log_freq	0.0098233	0.0002474	0.0000000	-0.0000920
preserved ~ CumErr+I(pos^2)+stimlen	0.0099153	0.0003394	0.0000920	0.0000000

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

69

```
##                                model p_accounted_for model_deviance diff_CumErr diff_CumErr+I(pos^2)
## 1                        preserved ~ CumErr          0.9744199      1870.386 0.0000000000      -0.0095759003
## 2                preserved ~ CumErr+I(pos^2)          0.9839958      1844.553 0.0095759000       0.0000000000
## 3 preserved ~ CumErr+I(pos^2)+stimlen+log_freq          0.9842432      1812.827 0.009823312       0.0002474113
## 4                preserved ~ CumErr+I(pos^2)+stimlen          0.9843352      1821.207 0.009915316       0.0003394154
## diff_CumErr+I(pos^2)+stimlen+log_freq diff_CumErr+I(pos^2)+stimlen
## 1                        -9.823312e-03                -9.915316e-03
## 2                        -2.474113e-04                -3.394154e-04
## 3                        0.000000e+00                -9.200414e-05
## 4                        9.200414e-05                 0.000000e+00
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```